DE LA IMAGEN A LA EXPERIENCIA

Visión con transformers y modelos multimodales

DOS EJEMPLOS LOCALES · ViT + Qwen3-VL

Imagen → categorías / Imagen + pregunta → texto

“Visión” no significa una sola cosa

TAREA	PREGUNTA	SALIDA
Clasificación · este ViT	¿Qué categoría describe la imagen?	Etiqueta + score
Detección	¿Dónde están los objetos?	Cajas + clases
Segmentación	¿Qué píxeles pertenecen a algo?	Máscara
Visión + lenguaje · este VLM	¿Qué hay y cómo lo describo?	Texto
Generación de imagen	¿Qué imagen quiero crear?	Píxeles nuevos

CNN y ViT organizan la información de maneras distintas

CNN clásica

Filtro local compartido → **Mapas de rasgos en varias capas** → **Contexto más amplio al combinar capas**

ViT clásico

Parches → vectores → **Atención entre todos los tokens** → **Representaciones contextualizadas**

Ambas aprenden con datos. La tarea depende del entrenamiento y de la salida.

Comparación conceptual · ViT original: arxiv.org/abs/2010.11929

La idea de token se conserva; la entrada y la tarea cambian

LLM de texto generativo

01 Texto → tokens del vocabulario

02 Embeddings + posición
Atención causal: contexto previo

03 Predice el siguiente token
Repite para generar texto

Este ViT clasificador

01 Imagen → parches de píxeles

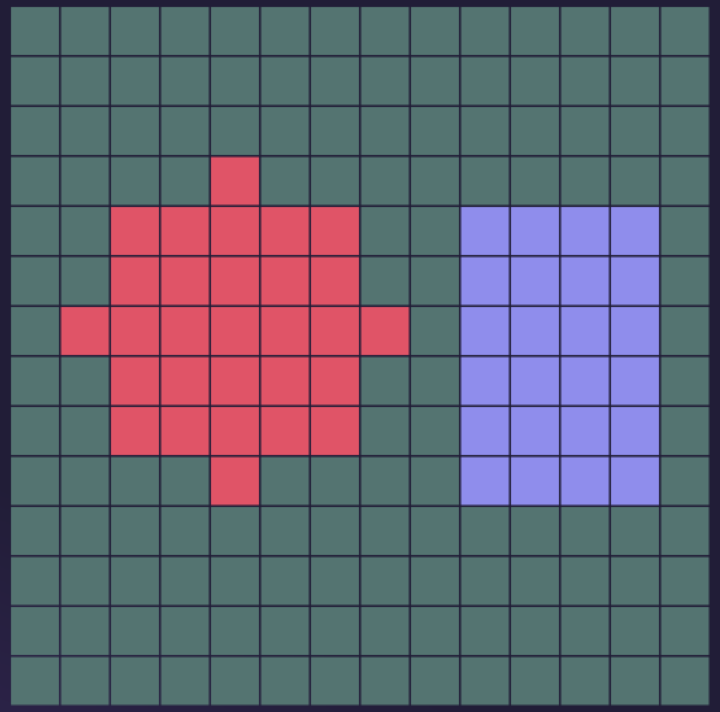
02 Embeddings + posición
Atención entre todos los parches

03 Representación final [CLS]
Cabeza → 1000 scores

ViT es un encoder visual. BERT también es un encoder, pero procesa texto.

Comparación con un LLM generativo causal · ViT Base: huggingface.co/google/vit-base-patch16-224

Una imagen se convierte en una secuencia



224 × 224 píxeles

16 × 16

píxeles por parche

14 × 14 = 196 parches

La cuadrícula muestra la entrada.
No muestra detección ni atención.

ViT: arxiv.org/abs/2010.11929 · checkpoint de Google

Cada parche recibe una representación y un lugar



196 parches + token [CLS] → secuencia de 197 posiciones en este ViT

huggingface.co/docs/transformers/model_doc/vit

Los parches intercambian información

MATRIZ ILUSTRATIVA

0.60	0.20	0.10	0.10
0.15	0.50	0.25	0.10
0.05	0.25	0.60	0.10
0.10	0.10	0.20	0.60

Una pieza ambigua puede apoyarse en otras piezas.

Atención = combinación de información.
No es una mirada consciente.

En OF, P dibuja parches: no esta matriz.

ViT: arxiv.org/abs/2010.11929

La atención calcula cuánto aporta cada representación

Q · consulta

¿Con qué información conviene relacionarme?

K · clave

¿Cómo se compara mi representación?

V · contenido

¿Qué información puedo aportar?

Q, K y V son proyecciones aprendidas de los mismos tokens.

$Q \times K^T / \sqrt{d}$
Comparar y escalar

→

softmax por fila
Obtener pesos

→

Pesos × V
Combinar información

Varias cabezas aprenden relaciones distintas. Atención ≠ explicación completa.

Esquema de una cabeza · ViT: arxiv.org/abs/2010.11929

El encoder repite atención y transformación por token

196 parches + [CLS]

197 tokens × 768 valores

UN BLOQUE DEL ENCODER

Normalizar + atención → **Sumar entrada (residual)** → **Normalizar + MLP** → **Sumar entrada (residual)**

Cada suma conserva también la entrada de su subbloque.

12 bloques

12 cabezas / bloque

Parámetros verificados en models/vit-base-patch16-224/config.json · Esquema pre-LayerNorm

El clasificador elige dentro de su vocabulario

[CLS] → cabeza de clasificación → logits → softmax



Datos didácticos. El top 5 no incluye todas las clases.

google/vit-base-patch16-224 · clasificación ImageNet

Descargamos lo aprendido; hacemos inferencia

ANTES · ENTRENAMIENTO

Imágenes + objetivos de aprendizaje

Ajuste de millones de parámetros

Resultado: pesos del modelo

AHORA · NUESTRA CLASE

Descargar pesos y preprocesador

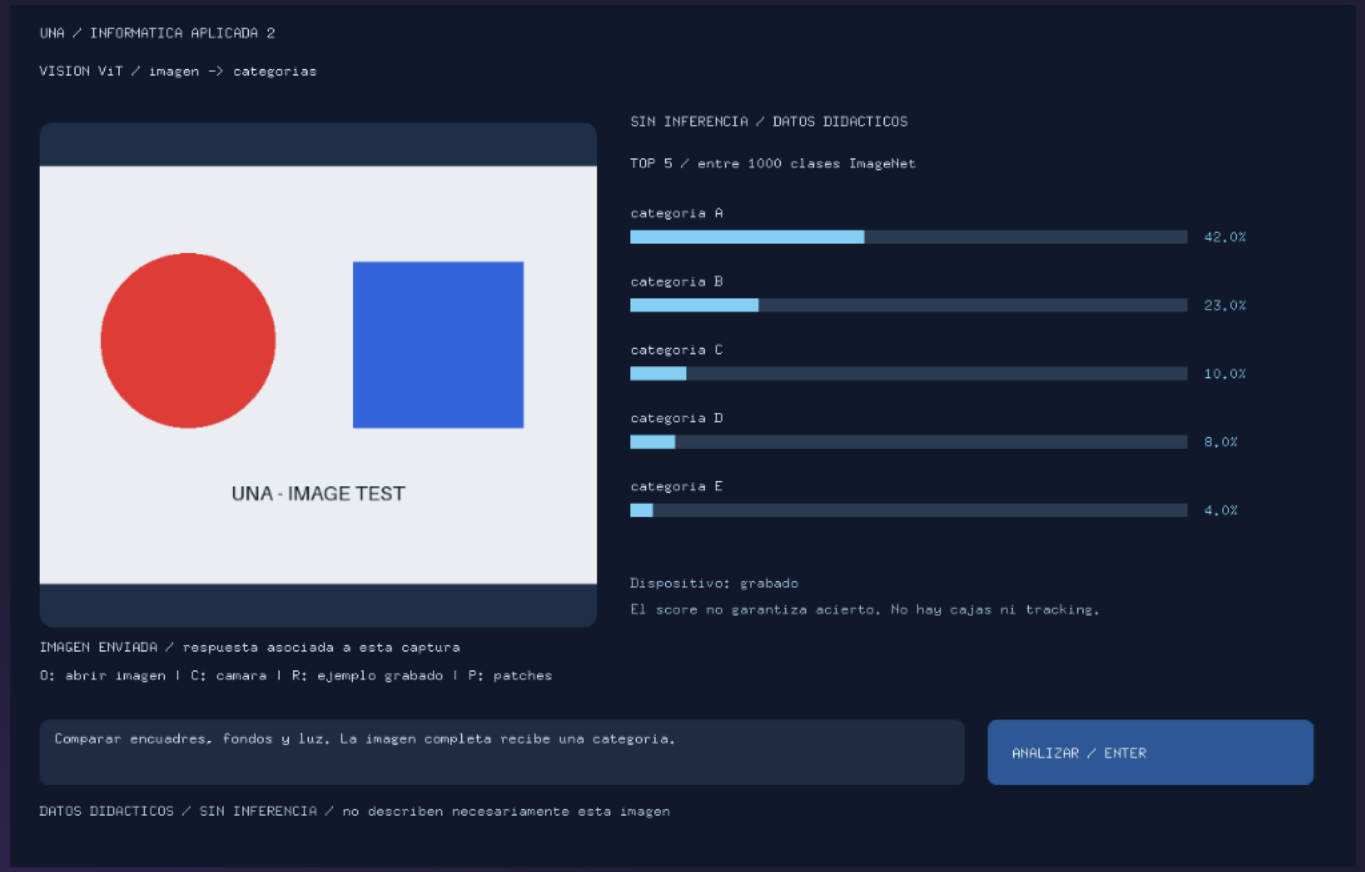
Enviar una imagen nueva

Usar la respuesta en OF

No entrenamos ViT ni Qwen durante esta práctica.

huggingface.co/google/vit-base-patch16-224

Una captura produce categorías, no coordenadas



01 O / arrastrar: foto

02 C: activar webcam

03 Enter: capturar y analizar

04 P: mostrar parches

05 Top 5: comparar scores

Interfaz real · datos grabados didácticos, sin inferencia

El encuadre también modifica la respuesta

01

Mismo objeto

Fotografiar un objeto.

02

Otro fondo

Mantener el objeto.

03

Otro encuadre

Cambiar distancia y recorte.

¿Cambió el objeto o cambiaron las pistas que usa el modelo?

Un modelo multimodal combina imagen y pregunta



La pregunta modifica qué pedimos sobre la imagen.

Qwen/Qwen3-VL-2B-Instruct · ficha del modelo

ViT y Qwen3-VL responden contratos diferentes

EN ESTA CLASE	visionViT	visionChat
Entrada	Una imagen	Imagen + pregunta
Procesamiento	Encoder visual + cabeza de clasificación	Encoder visual + integración con el modelo de lenguaje
Salida	Scores de categorías fijas	Texto generado
Uso desde OF	Mapear clase y score	Interpretar texto; validar un contrato si controla acciones

Un VLM puede usar un transformer visual: no son familias excluyentes.

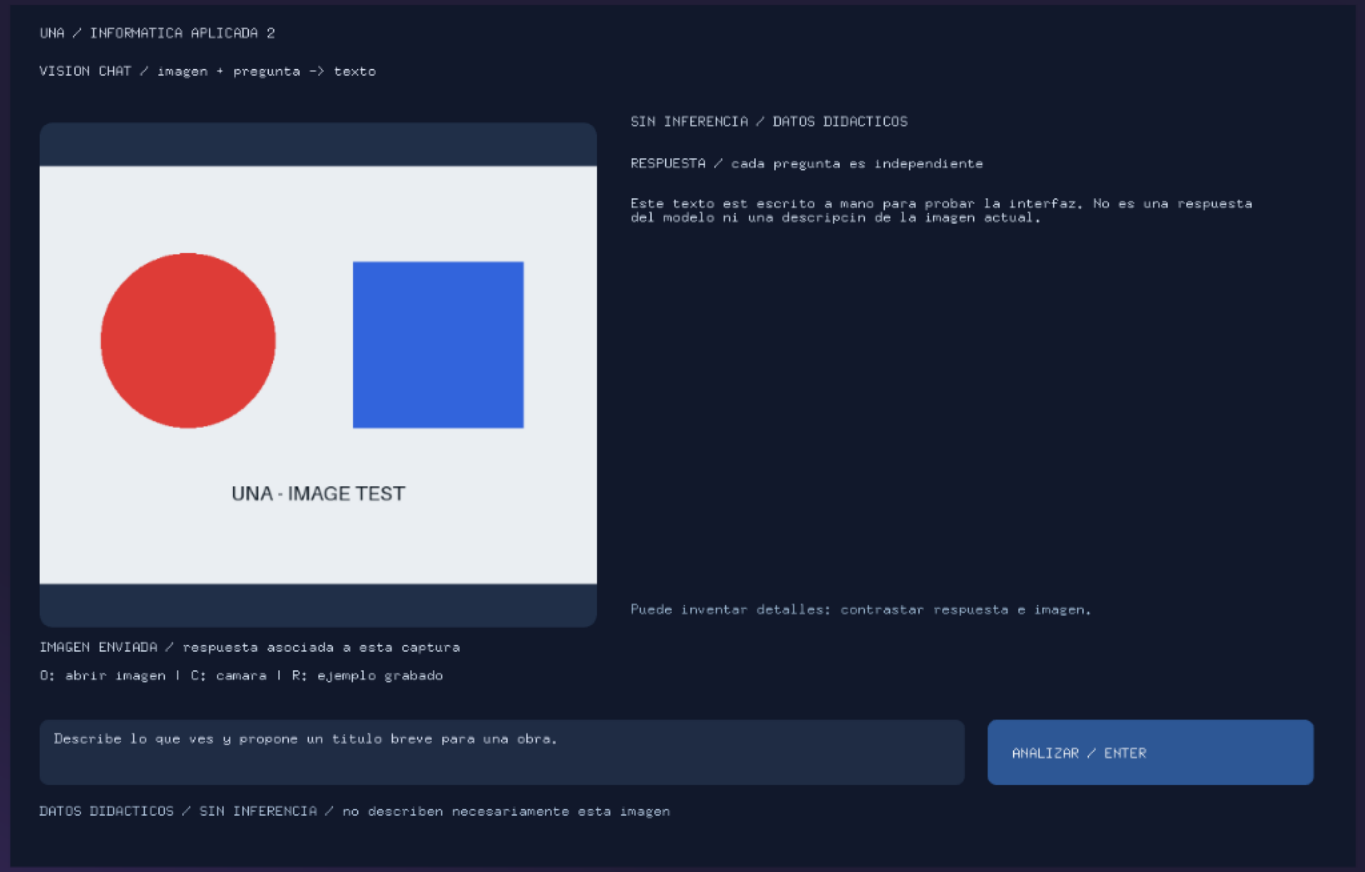
Checkpoints: google/vit-base-patch16-224 · Qwen/Qwen3-VL-2B-Instruct

NO SON SINÓNIMOS

Multimodal, cadena y generación son cosas distintas

SISTEMA	QUÉ COMBINA	EJEMPLO
Modelo multimodal	Representaciones de varias modalidades	Imagen + pregunta → texto
Cadena de modelos	Salidas de modelos independientes	Voz → transcripción → LLM
Generador de imagen	Condición → nuevos píxeles	Texto → imagen
Obra programada	Datos del modelo + reglas propias	Respuesta → escena OF / Box2D

La misma imagen admite preguntas diferentes



Interfaz real · texto grabado para probar el diseño

01 ¿Qué formas ves?

02 ¿Qué texto aparece?

03 ¿Qué no podés determinar?

04 Proponé un título

05 Contrastá con la imagen

Los ejemplos consultan servicios diferentes

ViT

visionViT
OF + addon → Python / PyTorch
127.0.0.1:8768 → Categorías
+ scores

MULTIMODAL

visionChat
OF + addon → Ollama / Qwen3-VL
127.0.0.1:11434 → Texto
generado

Enviar una imagen no es enviar su nombre

Mensaje para /api/chat

```
{
  "role": "user",
  "content": "¿Qué formas ves?",
  "images": ["JPEG_EN_BASE64"]
}
```

PÍXELES, NO UNA RUTA

Reducir imagen

Codificar JPEG

Convertir a base64

Enviar por HTTP

Respuesta: body.message.content

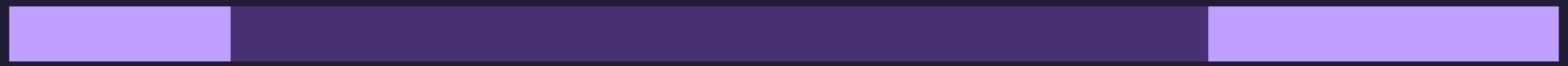
docs.ollama.com/capabilities/vision

La pantalla no debe esperar al modelo

OF / dibujo



Modelo / inferencia



Una petición por interacción. La animación continúa.

Cada motor decide dónde puede ejecutar

ViT · PYTORCH

CUDA si está disponible

Metal/MPS en Mac compatible

CPU si no hay aceleración viable

Qwen3-VL · OLLAMA

Gestiona sus propios pesos

Puede usar GPU, CPU o ambas

No usa el PyTorch del servidor ViT

Mismo protocolo desde OF; tiempos y memoria distintos según el equipo.

Un modificador agrega los dos modelos

macOS	bash install_env_mac.command --vision
Linux	bash install_env_linux.sh --vision
Windows	install_env_windows.cmd -Vision

.venv-vision separado · pesos locales · sin API key

Importar dos proyectos; reutilizar el addon

COPIAR

ofxLocalLLM → OF/addons/
visionViT → OF/apps/myApps/
visionChat → OF/apps/myApps/

IMPORTAR Y ACTUALIZAR

Project path: OF/apps/myApps
Project name: nombre del ejemplo
Addon: ofxLocalLLM
Additional source paths: vacío

ViT: abrir start_vision del sistema.
Multimodal: mantener Ollama abierto.

CONSIGNA DE TALLER

Transformen la respuesta en una decisión propia

01

Comparar

Misma imagen, dos modelos.
¿Qué salida sirve para tu obra?

02

Transformar

Cambiar el dibujo o mapear una respuesta a una acción.

03

Cuestionar

Registrar un caso útil y un caso que falle.

¿Qué decidió el modelo y qué programaron ustedes?

Modelos abiertos, decisiones visibles

ViT · arquitectura	arxiv.org/abs/2010.11929
ViT · pesos Apache 2.0	huggingface.co/google/vit-base-patch16-224
Qwen3-VL · pesos Apache 2.0	huggingface.co/Qwen/Qwen3-VL-2B-Instruct
Ollama · modelo exacto	ollama.com/library/qwen3-vl:2b-instruct
Ollama · imágenes en HTTP	docs.ollama.com/capabilities/vision

Sin cobro por consulta local. Hardware, memoria y evaluación siguen importando.